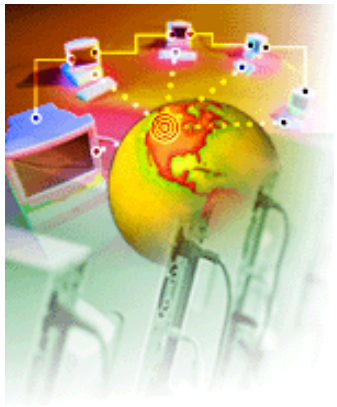
数 据 结 构

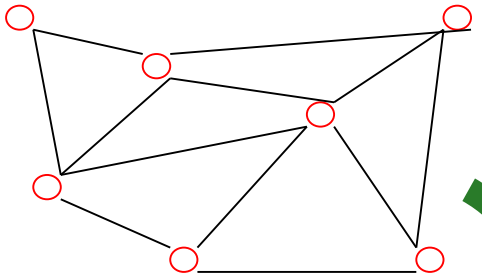
计算机科学与工程学院
陈强
chen2qiang@njust.edu.cn

课程意义



1001001111
ABCDEFG

姓名	学号	年龄
张三		
李四		
……		







“好”的程序

算法+**数据结构**=程序

内容安排

第1章 绪论	2 课时
第2章 线性表	5 课时
第3章 栈和队列	4 课时
第5章 数组与广义表	4 课时
第6章 树与二叉树	7 课时
第7章 图	7 课时
第9章 查找	7 课时
第10章 内部排序	4 课时
上机实验（线性表 4，二叉树或图 4）	8 课时

考核方法

- 平时成绩 10%
  - 考勤
  - 作业
- 上机实验 10% （程序+实验报告）
  - 线性表链式应用
  - 二叉树有关运算
  - 图有关运算
- 期末考试成绩 80% （闭卷笔试）

课程信息

- 教材：张琨、张宏、朱保平编著. 数据结构与算法分析（C++语言版），2020年5月第二版.

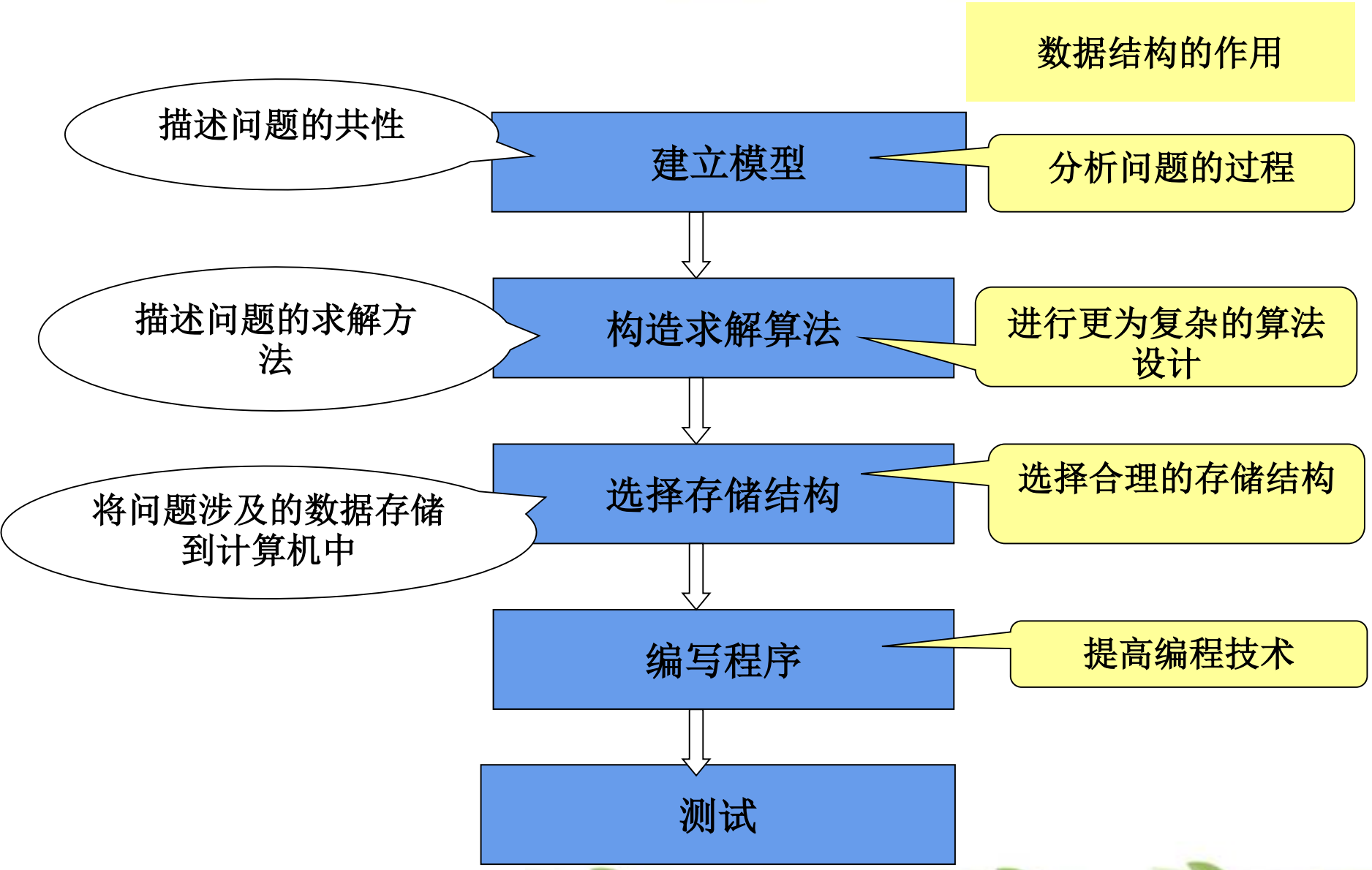
  严蔚敏,吴伟民编著. 数据结构. 清华大学出版社（C语言版）, 2013年4月第三版.

- 先修课程：C++程序设计

第一章　　绪　论

- 1.1 什么是数据结构
- 1.2 基本概念和术语
- 1.3 抽象数据类型的表示与实现
- 1.4 算法和算法分析

1.1 什么是数据结构



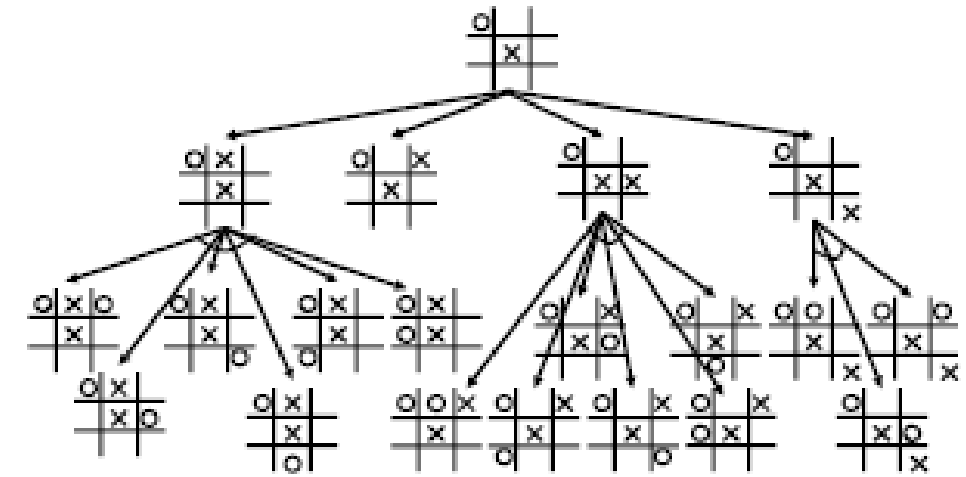
用计算机解决问题的过程

1.1 什么是数据结构

数据结构是一门研究非数值计算的程序设计问题中计算机的操作对象以及它们之间的关系和操作等的学科。

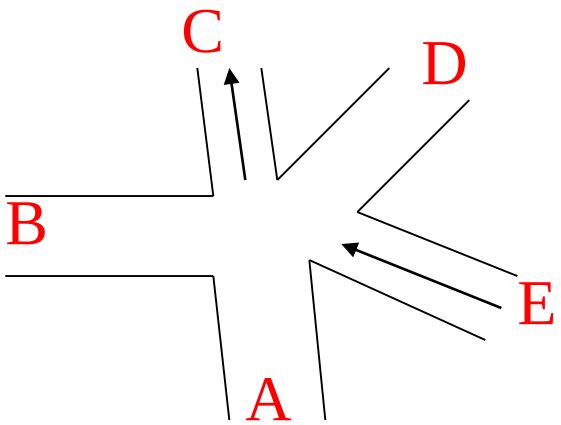
例1.1 图书馆书目检索系统——线性（书p.1-2）

例1.2 计算机和人对弈问题——树型（书p.1-2）

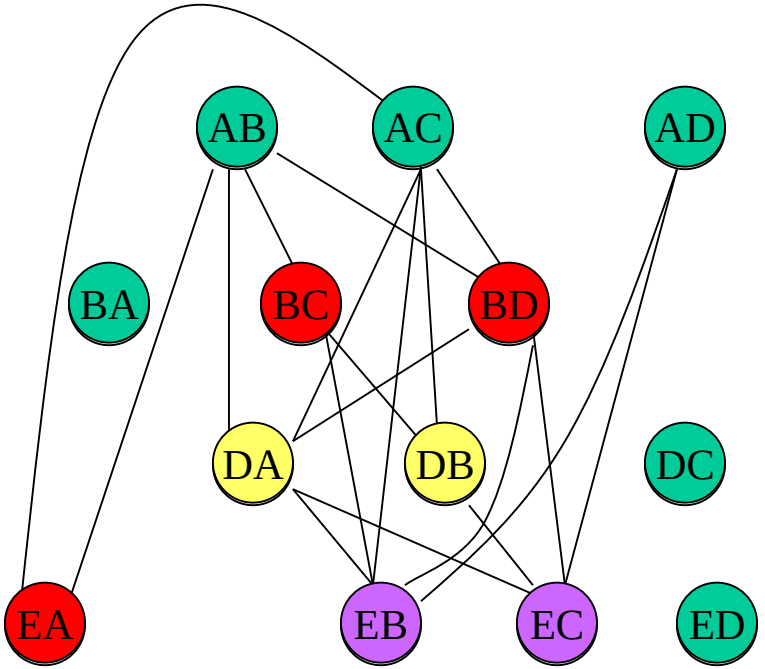


棋局构成一棵"树" ----博弈树

例1.3 多叉路口交通灯管理系统



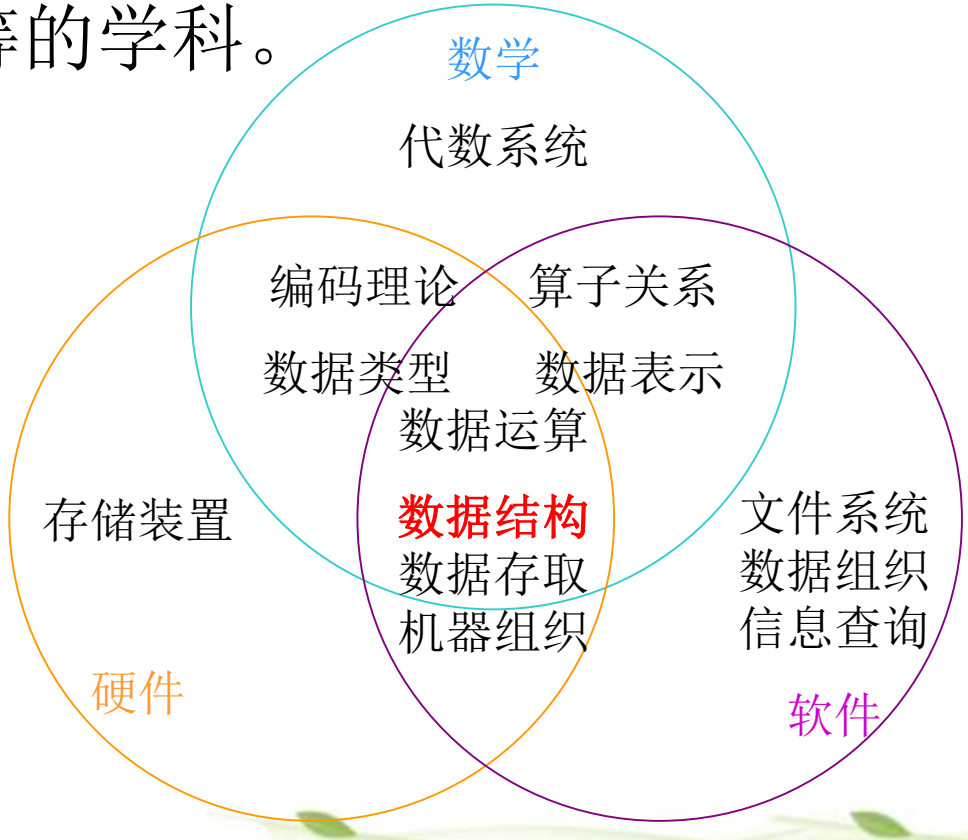
13条通路，考察任意两条通路是否互相碰撞，在78种情况下有20种情况会碰撞（用连线表示）



设置交通灯的问题等价于对图的顶点着色问题：要求对图上的每一个顶点染一种颜色，并且要求有线相连的两个顶点不能具有相同颜色，而总的颜色种类应尽可能地少。

数据结构课程

- 数据结构是一门研究非数值计算的程序设计问题中计算机的操作对象以及它们之间的关系和操作等等的学科。

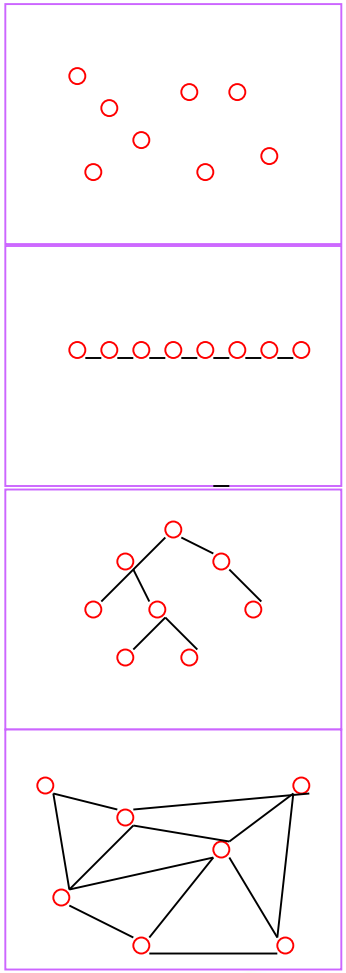


1.2 基本概念与术语

- **数据(Data):**是对信息的一种符号表示。在计算机科学中是指所有能输入到计算机中并被计算机程序处理的符号的总称。
- **数据元素(Data Element):**是数据的基本单位，在计算机程序中通常作为一个整体进行考虑和处理。一个数据元素可由若干个**数据项(Data Item)**组成。数据项是数据的不可分割的最小单位。
- **数据对象(Data Object):**是性质相同的数据元素的集合。是数据的一个子集。
- **数据结构(Data Structure):**是相互之间存在一种或多种特定关系的数据元素的集合。

数据的逻辑结构

- 数据元素之间的相互关系称为逻辑结构。通常分为四类基本结构：
  - **集合**：结构中的数据元素除了同属于一种类型外，别无其它关系
  - **线性结构**：结构中的数据元素之间存在一对一的关系
  - **树型结构**：结构中的数据元素之间存在一对多的关系
  - **图状结构或网状结构**：结构中的数据元素之间存在多对多的关系



数据结构的形式定义

数据结构的形式定义为：数据结构是一个二元组

Data_Structure =(D，S)

D是数据元素的有限集

S是D上关系的有限集

例1.4 在计算机科学中，复数可取如下定义：复数是一种数据结构：Complex = （C，R）

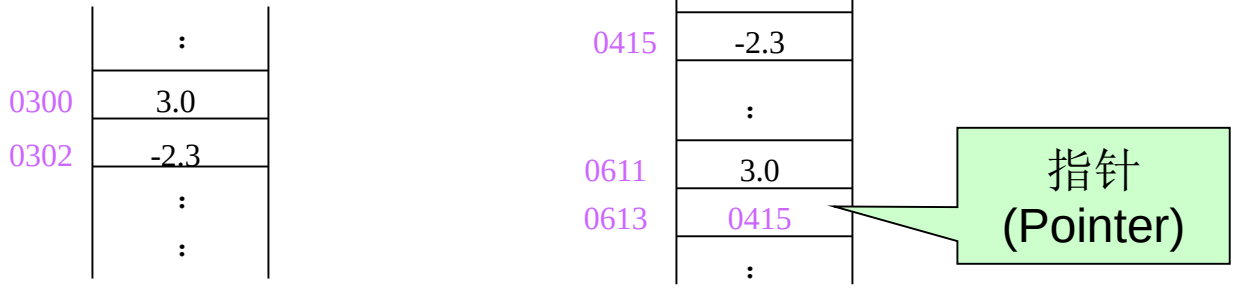
C是含两个实数的集合{c1，c2}

R ={<c1,c2>}，这里有序偶<c1,c2>表示c1是复数的实部，c2是复数的虚部

数据的存储结构

- 数据结构在计算机中的表示称为数据的存储结构或数据的物理结构。

例：复数z=3.0-2.3i的两种表示见下图。



顺序映象　　非顺序映象

顺序存储结构　　链式存储结构

数据类型

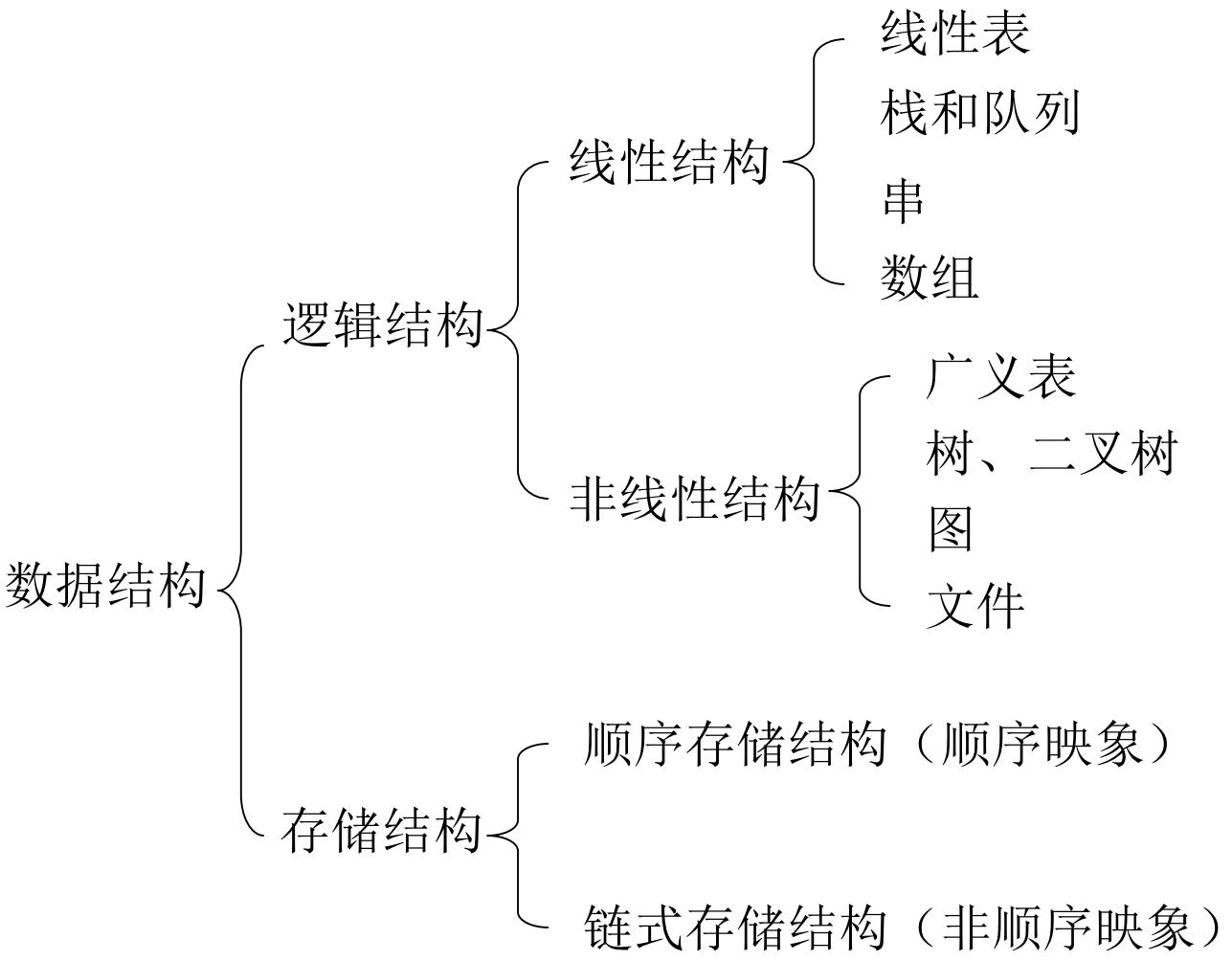
- 数据类型就是在程序设计语言中，变量所具有的数据种类。换句话说，数据类型是一个值的集合和定义在这个值集上的一组操作的总称。

- 例如：在FORTRAN语言中，变量的数据类型有整型、实型、和复数型
- 例如：在C++语言中，数据类型：基本类型和构造类型

数组、结构、联合、指针、自定义

整型、浮点型、字符型

数据结构的分类


数据结构
逻辑结构
线性结构
线性表
栈和队列
串
数组
非线性结构
广义表
树、二叉树
图
文件
存储结构
顺序存储结构（顺序映象）
链式存储结构（非顺序映象）

1.3 抽象数据类型的表示与实现

- **抽象数据类型(Abstract Data Type简称ADT)**是指一个数学模型以及定义在该模型上的一组操作。
- 抽象数据类型可用以下三元组表示

  ADT =（D，S，P）

  - D是数据对象
  - S是D上关系的有限集
  - P是对D的基本操作集。

抽象数据类型三元组

例：

```
ADT Triplet{
    数据对象：D={e1, e2, e3|e1,e2,e3∈ElemSet}
    数据关系：R1={<e1, e2>, <e2, e3>}
    基本操作：
            InitTriplet(&T, v1, v2, v3);
            DestroyTriplet(&T);
            Get(T, i, &e);
            Put(&T, i, e);
            IsAscending(T);
            IsDescending(T);
            Max(T, &e);
            Min(T, &e);
}ADT Triplet
```

线性结构

类C语言

- 可以采用介于伪码和C语言之间的类C语言作为抽象数据类型的描述工具。这使得数据结构和算法的描述和讨论简明清晰，不拘泥于C语言的细节，又能够容易转换成C或者C++程序。
- 类C语言精选了C语言的一个核心子集，同时作了若干扩充修改，增强了语言的描述功能。
- 关于类C语言见教材p10-p13

1.4 算法和算法分析

- **算法：**是对特定问题求解步骤的一种描述，是指令的有限序列，其中每一条指令表示一个或多个操作。
- 算法具有以下五个特性：
  - **有穷性** 一个算法必须总是在执行有穷步之后结束，且每一步都在有穷时间内完成。
  - **确定性** 算法中每一条指令必须有确切的含义。不存在二义性。
  - **可行性** 一个算法是可行的。即算法描述的操作都是可以通过已经实现的基本运算执行有限次来实现的。
  - **输入** 一个算法有零个或多个输入，这些输入取自于某个特定的对象集合。
  - **输出** 一个算法有一个或多个输出，这些输出是同输入有着某些特定关系的量。

算法设计的要求

- **正确性** 算法应满足具体问题的需求。“正确”有四个层次：

  (a)不含语法错误；(b)对几组输入正确；

  (c)对精心设计的测试输入正确；(d)对一切合法输入正确
- **可读性** 算法应该好读。以有利于阅读者对程序的理解。
- **健状性** 算法应具有容错处理。当输入非法数据时，算法应对其作出反应，而不是产生莫名其妙的输出结果。
- **效率与存储量需求** 效率指的是算法执行的时间；存储量需求指算法执行过程中所需要的最大存储空间。一般，这两者与问题的规模有关。

算法效率的度量

- 算法执行时间需通过依据该算法
上运行时所消耗的时间来度量。
  - **事后统计的方法** 收集此算法的执
计资料
  - **事前分析估算的方法** 求出该算法的

例：for(i=1;i<=n;++i)

for(j=1;j<=n;++j){

c[i][j]=0;

for(k=1;k<=n;++k)

c[i][j]+=a[i][k]*b[k][j];

}

一个算法是由控制结构（顺序、分支和循环三种）和原操作（指固有数据类型的操作）构成的，则算法时间取决于两者的综合效果。为了便于比较同一问题的不同算法，通常的做法是，从算法中选取一种对于所研究的问题来说是基本操作的**原操作**，以该基本操作重复执行的次数作为算法的时间量度。

撇开与计算机硬件、软件有关的因素，可以认为一个特定算法“运行工作量”的大小只依赖于问题的规模（通常用整数量n表示）。

渐近时间复杂度

- 一般情况下，算法中基本操作重复执行的次数是问题规模n的某个函数，算法的时间量度记作

  T(n)=O(f(n))

  称作算法的**渐近时间复杂度(Asymptotic Time Complexity)**。

- 定义：如果g(n)是正整数n的一个函数，若存在两个正常数c和n_0，对于所有的$n \geq n_0$，有$|g(n)| \leq c\,|f(n)|$，则记作 g(n)=O(f(n))。

频度

频度是指语句重复执行的次数。

例1　{++x; s=0;}

将x自增看成是基本操作，则语句频度为１，即时间复杂度为Ｏ(1)。

如果将s=0也看成是基本操作，则语句频度为1，其时间复杂度仍为Ｏ(1)，即常量阶。

例2　for(i=1;i<=n;++i)

{++x; s+=x;}

语句频度为：n　其时间复杂度为：O(n)，　即时间复杂度为线性阶。

时间复杂度举例

例3
```
for(i=1;i<=n;++i)
        for(j=1;j<=n;++j)
                {++x;  s+=x;}
```

语句频度为：n^2，其时间复杂度为：$O(n^2)$，即时间复杂度为平方阶。

例4
```
for(i=0;i<=n-1;++i)
        for(j=0;j<=i;++j)
                a[i][j]=0;
```

时间复杂度为：$O(n^2)$

i=0: 赋值 1 次

i=2: 赋值 3 次

……..

+

i=n-1: 赋值 n 次

$1+2+3+\ldots+n=(1+n)n/2=n^2/2+n/2$

练习

- **1.** 下面程序的时间复杂度为 O(m*n) 。

```
for(i = 0; i < m; i++)
  for(j = 0; j < n; j++)
    A[i][j] = i * j;
```

- **2.** 下面程序的时间复杂度为 O(n) 。

i = 1; while(i <= n) i = i +2;

练习

- **3.** 提高题。 $O(\log_2 n)$

```
void hanoi(int n, char x, char y, char z)
{  if (n ==1)
      cout<<x<<z;
   else
   {
      hanoi(n-1, x, z, y);
      cout<<x, z;
      hanoi(n-1, y, x, z);
    }
}
```

$n = 1, T(1) = 1$

$n > 1, T(n)=2T(n-1)+1$

i=n-1: j从0到n-2 交换n-1次

i=n-2: j从0到n-3 交换n-2次

i=n-3: j从0到n-4 交换n-3次

……

\+ i=1: j从0到0 交换1次

$(n-1)+(n-2)+(n-3)+\ldots+1=(n-1)n/2=n^2/2-n/2$

时间复杂度的不同量级

- 最常用的六种多项式时间及其关系为：
  $O(1)<O(\log n)<O(n)<O(n\log n)<O(n^2)<O(n^3)$
- 最常用的三种指数时间及其关系为：
  $O(2^n)<O(n!)<O(n^n)$
- 当n取得很大时，指数时间算法和多项式时间算法在所需时间上非常悬殊（参见书p16图1.7）。

最坏时间复杂性

- 有的情况下，算法中基本操作重复执行的次数还随问题的输入数据集不同而不同。

例如：

```
void  bubble-sort(int a[]，int n)
      for(i=n-1;i>0 ;i--)
            for(j=0;j<i;++j)
                if (a[j]>a[j+1])
                   a[j] ←→a[j+1];
```

最好情况：0次　（输入1,3,5…升序序列）

最坏情况：每次都换，　$O(n^2)$ （输入5,3,1…降序序列）

空间复杂度

- 空间复杂度:算法所需存储空间的度量，记作:

  $S(n)=O(f(n))$

  其中n为问题的规模(或大小)

- 时间与空间往往是对矛盾，要综合考虑。

本章学习要点

- 熟悉各名词、术语的含义，掌握基本概念。
- 理解算法五个要素的确切含义。
- 掌握计算语句频度和估算算法时间复杂度的方法。

作业1

1.1 试分析时间复杂度

```
sum=1;
for(i=0; sum<n; i++)
    sum+=i;
```

1.2 试给出一个时间复杂度为$O(\log_2(n))$的程序段。